Multithreaded algorithms

Dynamic multithreading uses a dependency DAG to separate total work, critical path and scheduling. Starting from spawn, sync and race avoidance, this chapter derives matrix multiplication and parallel merge/sort, with correctness proofs and space assumptions.

Prerequisites are DAGs, recurrence analysis and parallel cost models. We also distinguish a parallelism bound from the actual benefit of another processor.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The basics of dynamic multithreading

Programs specify computations allowed to run concurrently; the scheduler decides when and on which processor they run. A logical task is not a permanently bound OS thread, and spawning does not promise an immediate second core.

Assume P identical ideal processors and sequentially consistent shared memory. Instructions interleave in an order respecting dependencies; each load/store is one step, but a combined read-add-write is not automatically atomic. Arithmetic, indexing, and spawn/sync have constant unit work. Scheduler overhead, bandwidth, caches, and communication are outside this ideal bound.

A nonempty finite computation is a DAG. Nodes can be strands, sequential instruction sequences without parallel control, with positive execution time. Remove empty strands; split positive integral strands into unit-instruction chains for unit-node proofs. Edges express control and data dependencies. Nodes merely drawn at the same level need not be independent.

Exact semantics of spawn, call, and sync

`x = spawn F(...)` allows F to run with the caller's continuation. An ordinary `y = G(...)` requires the caller to await G's return. A `sync` waits for all children previously spawned by this procedure; it is not a system-wide barrier.

Spawn arguments are evaluated in the parent first. Each child owns its call-local variables, but shared-array references can still lead several tasks to the same cell. Different variable names do not prove independence. Reading a spawned result requires a dependency, usually an explicit sync.

Procedure return has an implicit sync here. However, if `return x+y` evaluates its expression before returning, that implicit wait does not permit an earlier read of x. Explicitly sync before computing the return expression. Removing spawn/sync and replacing parallel for by ordinary for gives a serialization. Correct serialization alone does not prove every parallel interleaving correct.

Fibonacci code and correctness

```
P-FIB(n):                // nonnegative integer n
    if n <= 1: return n
    x = spawn P-FIB(n-1)
    y = P-FIB(n-2)
    sync
    return x+y
```

Children read their own n and return separate results, with no shared writes. Decreasing n ensures termination. Bases F0=0, F1=1 and induction, followed by synchronized addition, prove Fn. Spawning both children still requires waiting before addition and changes constants rather than work/span orders.

This version deliberately omits memoization and recomputes parameters. High parallelism does not imply work efficiency: sequential iteration uses O(n) arithmetic operations, whereas occupying more cores does not excuse exponential work.

Work and span in one explicit strand model

Split each nonbase call into three unit strands:pre through spawning, cont before the second ordinary call, and join for addition/return after sync. A base call is one unit strand. Pre precedes the left child and cont; cont precedes the right child; both child finishes precede join.

With total work W and longest dependency-path cost D,

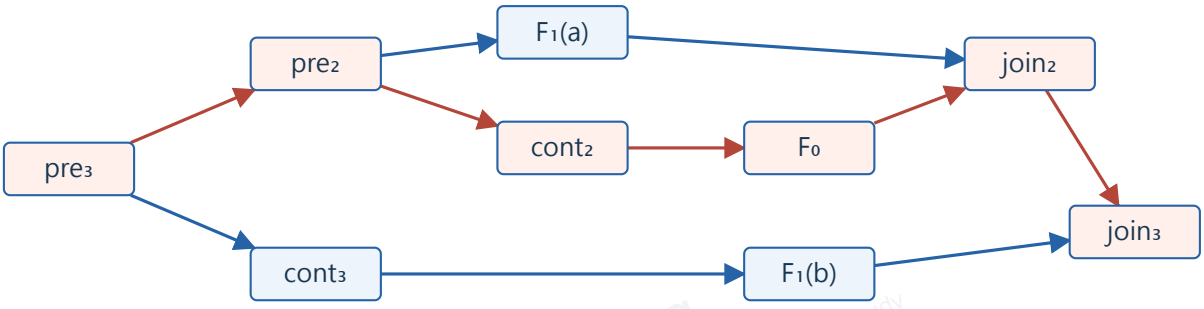
$$W_n = W_{n-1} + W_{n-2} + 3, \qquad W_0 = W_1 = 1;$$
$$D_n = 2 + \max(D_{n-1}, 1 + D_{n-2}), \qquad D_0 = D_1 = 1.$$

n	Fn	Wn	Dn
2	1	5	4
3	2	9	6
4	3	17	8

Induction verifies Wn=4F(n+1)−3 and Dn=2n for n≥2. Let $\varphi = (1+\sqrt{5})/2$. Since ϕ²=ϕ+1, induction gives ϕ^(n−2)≤Fn≤ϕ^(n−1) for n≥2:check2,3, and add adjacent bounds to obtain the next power. Thus W=Θ(ϕ^n), D=Θ(n). A complete recursive call is not one O(1) node in this model.

Trace all dependencies among nine strands

All nine strands of P-FIB(3): arrows express dependencies



Each box is one unit strand: W=9. The red six-box critical path gives D=6.

Pre3 forks to pre2 and cont3; cont3 calls the right F1. Pre2 forks to its F1 and cont2; cont2 calls F0. Both bases precede join2, and join2 plus the other F1 precede join3.

The red path pre3→pre2→cont2→F0→join2→join3 has 6 nodes, so W=9, D=6. Path pre3→cont3→F1→join3 has only 4. Arrows express dependencies, not a single sequential chain through every node of a call.

Work law, span law, and achievable speedup

Write $W = T_1$,$D = T_\infty$, and actual P-processor time T_P. At most P units of work finish per time unit, and a critical path cannot parallelize, so

$$T_P \geq \max\{W/P, D\}.$$

The discrete unit-node model also gives $T_P \geq \lceil W/P \rceil$. Speedup $S_P = W/T_P \leq \min(P, W/D)$, and efficiency S_P/P≤1. Average parallelism A=W/D limits speedup. P>A precludes perfect P-fold acceleration, but does not imply processor P+1 has no benefit.

Sequential composition adds both W and D. Independent fork/join branches add work and take the maximum branch span, plus explicit fork/join costs. An empty computation has W=D=0 and time 0;0/0 is not defined parallelism.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete greedy-scheduling upper-bound proof

At each step execute P ready nodes if possible, otherwise execute all ready nodes. Let C be complete steps and I incomplete steps, T=C+I. Every incomplete step before completion executes at least one node, so W≥PC+I.

An incomplete step removes all sources of the remaining DAG. Any longest path remaining afterward can be extended backward in the previous DAG to at least one removed source. If nothing remains, the earlier span was at least 1. Thus each incomplete step decreases remaining span by at least 1, and I≤D. Consequently

$$T \leq (W - I)/P + I \leq (W - D)/P + D \leq W/P + D.$$

The substitution uses coefficient1−1/P≥0. Positive integral strand durations can be split into chains for this proof. It does not give the same guarantee in a different model with arbitrary indivisible tasks of unknown lengths.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Two-approximation and quantified slackness

An optimal scheduler also obeys $T_P^* \geq \max(W/P, D)$, so greedy T≤W/P+D≤2T*. The sharper bound is(2−1/P)T*. Constant-factor performance requires no future-DAG knowledge.

Define slackness $\sigma = W/(PD) = A/P$. For D>0, the bounds give

$$1 \leq T_P/(W/P) \leq 1 + (1 - 1/P)/\sigma \leq 1 + 1/\sigma.$$

Large σ brings ideal greedy time near W/P. This is a mathematical guarantee, not measured bandwidth or scheduler overhead. An optimal schedule can ignore excess processors, so its time cannot increase with P. Measured implementations also depend on overhead and scheduling choices.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Parallelism 30 still benefits from processor 31

Complete 697 independent unit tasks before starting a 23-unit dependency chain. Then W=697+23=720, D=1+23=24, A=30.

P	Independent-stage allocation	Stage time	Chain time	Exact optimum
30	23 rounds of 30, then7	24	23	47
31	22 rounds of 31, then 15	23	23	46

The chain cannot start until every predecessor finishes. Thus ceil(697/P)+23 is both an attainable schedule and a lower bound. Speedups 720/47≈15.32 and 720/46≈15.65 both stay below 30, yet the added processor helps. Only P≥697 makes the first stage one round and attains span 24.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

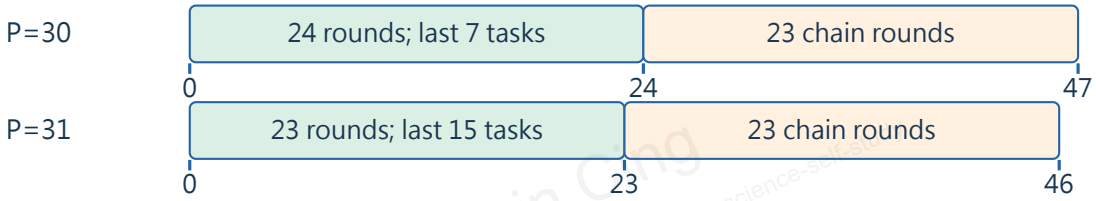
Parallelism 30 still benefits from processor 31 (continued)

Same DAG: processor 31 still improves on 30



W=697+23=720
D=1+23=24

Average parallelism W/D=30. Both rows use the same time scale.



$T_{30}=47$, $T_{31}=46$. Speedup≤30 does not preclude benefit from processor 31.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Less work but greater span can reverse the winner

Two correct versions have(W, D)=(1024,4) and(768,16), both with ideal greedy scheduling.

At P=8, the first takes at least 1024/8=128; the second takes at most 768/8+16=112, so the second is definitely faster. At P=128, the first takes at most 1024/128+4=12; the second takes at least 16, so the first is faster.

Disjoint upper/lower bounds justify these conclusions. W/P+D is an upper bound, not exact time. Overlapping feasible-time intervals may leave the ordering unresolved from W, D alone. A single small-core measurement also cannot identify separate work and span changes.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Implementing parallel for by balanced fork/join

Use zero-based half-open intervals[l, r), with l=r empty, here and in merge algorithms.

```
PAR-FOR(l, r, body):
    if l == r: return
    if r-l == 1: body(l); return
    m = floor((l+r)/2)
    spawn PAR-FOR(l, m, body)
    PAR-FOR(m, r, body)
    sync
```

The n nonempty iterations give n leaves and n−1 internal nodes. Constant control work per internal node adds O(n), not n log n. The longest call/join chain has depth Θ(log n), so span is Θ(log n) plus the largest iteration span, provided iterations are independent.

An ordinary loop spawning n tasks serially has a Θ(n) issuing chain in its parent and cannot claim the same logarithmic span. The final sync also makes all results ready before the next phase reads them.

Matrix-vector multiplication safely parallelizes rows

```
MAT-VEC(A, x):
    n=A.rows
    allocate y[n]
    parallel for i=0 to n-1:
        y[i]=0
        for j=0 to n-1: y[i]+=A[i, j]*x[j]
    return y
```

Iteration i exclusively owns y[i], with A, x read-only. After inner step j, the entry is the sum of the first j+1 products, proving correctness. Work is $\Theta(n^2)$, span Θ(log n)+Θ(n)=Θ(n), parallelism Θ(n). For n=0 return an empty vector.

Making the inner loop parallel as well causes unsynchronized read-modify-writes to one y[i], a determinacy race. Two logically parallel accesses conflict when they touch one cell and at least one writes. Even equal written values or one successful run does not prove independence.

Sequential consistency still permits lost updates

Lost updates: loads, local registers, and final stores

Initially x=0. Each task preserves load → add → store.

Step	Task 1	Task 2	Shared x
1	load $r_1 \leftarrow x = 0$		0
2		load $r_2 \leftarrow x = 0$	0
3	add $r_1 \leftarrow r_1 + 1 = 1$		0
4	store $x \leftarrow r_1$		1
5		add $r_2 \leftarrow r_2 + 1 = 1$	1
6		store $x \leftarrow r_2$	1

After sync both read 1. Waiting cannot make three instructions one atomic operation.

The interleaving respects each task's load→add→store but ends at x=1. Running one task completely before the other gives 2. Sequential consistency orders individual instructions; it does not make the three-instruction compound operation atomic.

Use disjoint ownership or reduction to remove the race. Atomic additions or locks can impose dependencies, but their serialization, work, and span need separate accounting. They do not retain an analysis assuming all additions complete simultaneously.

A reduction parallelizes the dot product correctly

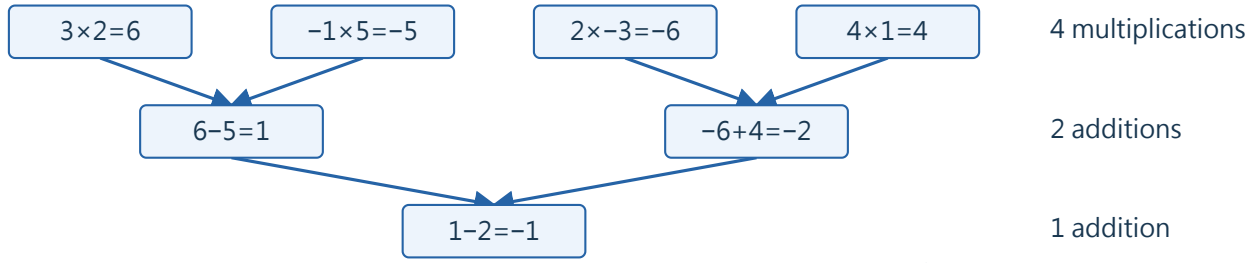
```
DOT(Arow, x, l, r):
    if l == r: return 0
    if r-l == 1: return Arow[l]*x[l]
    m=floor((l+r)/2)
    left=spawn DOT(Arow, x, l, m)
    right=DOT(Arow, x, m, r)
    sync
    return left+right
```

Children return separate values rather than sharing an accumulator. Associativity and induction give the interval dot product. Work is $\Theta(n)$, span $\Theta(\log(n+1))$. Replacing each MAT-VEC row by DOT gives $\Theta(n^2)$ work and $\Theta(\log n)$ span for $n \geq 2$.

For row=[3,−1,2,4], x=[2,5,−3,1], products are[6,−5,−6,4], first-level sums[1,−2], and result−1.

A reduction parallelizes the dot product correctly (continued)

Dot product: four private products combine in a tree



Arithmetic DAG: work=7, span=3. Fork/join adds Θ(log n) control depth.

Floating-point tree addition may round differently from sequential accumulation. The proof assumes exact associative arithmetic and does not promise bitwise-identical floating results.

Granularity and the allocation model

Group n constant-cost iterations into batches of at most g,$1 \le g \le n$, and issue batches with balanced PAR-FOR. Work is Θ(n), span $\Theta(\log\lceil n/g \rceil + g)$. Larger g reduces task overhead constants but increases leaf serial time; the tradeoff depends on P and slackness.

Here allocate obtains a descriptor for uninitialized, nonoverlapping storage at constant abstract span. If actual allocation initializes each cell, count that work and parallelize initialization to retain logarithmic span. Serial zeroing of n^2 cells is not hidden inside O(1).

Matrix blocks use views(row origin, column origin, stride, order), without hidden full-block copies. Task descriptors, allocators, caches, and memory bandwidth have implementation costs beyond the stated ideal model.

Multithreaded matrix multiplication

```
P-MULTIPLY(A, B):
    n=A.rows                // both n by n
    allocate C[n, n]
    parallel for i=0 to n-1:
        parallel for j=0 to n-1:
            C[i, j]=0
            for k=0 to n-1: C[i, j]+=A[i, k]*B[k, j]
    return C
```

Each(i, j) owns one output cell and accumulates its inner loop sequentially. Work is $\Theta(n^3)$. The two issuing trees contribute $\Theta(\log n)+\Theta(\log n)$, then the inner loop contributes $\Theta(n)$, so span $\Theta(n)$, parallelism $\Theta(n^2)$. Two parallel keywords do not remove each cell's n-term dependency.

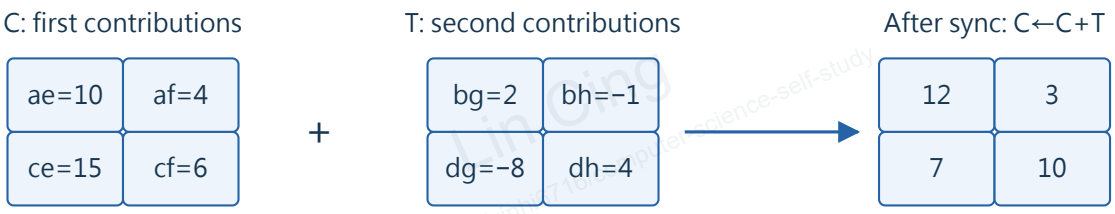
Using a DOT-style reduction for each Cij preserves $\Theta(n^3)$ work but reduces span to $\Theta(\log n)$, giving $\Theta(n^3/\log n)$ parallelism. Concurrent writes to Cij are not this reduction algorithm.

Eight subproducts need eight distinct destinations

Write A=[[a, b],[c, d]], B=[[e, f],[g, h]], where letters are equal-order blocks. Compute C=[[ae, af],[ce, cf]] and temporary T=[[bg, bh],[dg, dh]]. Every one of eight products has its own destination. Sync before C=C+T.

For A=[[2,−1],[3,4]], B=[[5,2],[−2,1]], temporary C=[[10,4],[15,6]], T=[[2,−1],[−8,4]], giving[[12,3],[7,10]].

Eight independent products: separate destinations before addition



Each product writes one C or T cell. Await all products, then add cellwise.
A=[[2,−1],[3,4]], B=[[5,2],[−2,1]]. No simultaneous += contributions to one cell.

Two tasks executing C11+=ae and C11+=bg conflict. The temporary storage removes a shared-write dependency; spawning alone cannot do that.

Complete recursive matrix-product procedure

```
P-REC(C, A, B):
    n=A.rows
    if n == 1: C[0,0]=A[0,0]*B[0,0]; return
    allocate T[n, n]
    view A as [[a, b],[c, d]], B as [[e, f],[g, h]]
    view C, T as four n/2 by n/2 blocks
    spawn P-REC(C11, a, e); spawn P-REC(C12, a, f)
    spawn P-REC(C21, c, e); spawn P-REC(C22, c, f)
    spawn P-REC(T11, b, g); spawn P-REC(T12, b, h)
    spawn P-REC(T21, d, g); P-REC(T22, d, h)
    sync
    parallel for i=0 to n-1:
        parallel for j=0 to n-1: C[i, j]+=T[i, j]
```

Inputs are equal-order square matrices of power-of-two order, A, B read-only, and C disjoint from them. Pad any other positive order with zeros to the next power of two and extract the original result. Return an empty matrix for n=0. T and child workspaces are independent. Eight spawn/call operations are a fixed constant, not an n-long issuing chain.

Inductive correctness and eight-way costs

The base writes the correct scalar product. Inductively, eight half-size products are correct and race-free through distinct destinations. Sync makes them available before cellwise addition. Block multiplication identities then yield AB. Shared reads of A, B are harmless.

$$W_M(n) = 8W_M(n/2) + \Theta(n^2) = \Theta(n^3),$$
$$D_M(n) = D_M(n/2) + \Theta(\log n) = \Theta(\log^2 n).$$

Span follows only the longest child, not the sum of eight. Each level's nested parallel combination has Θ(log n) issuing/join depth, not O(1). For n=2^L, the sum1+…+L=Θ(L^2) gives both bounds. Order4 has 8^2=64 scalar-product leaves; parallelization has not removed multiplications.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Three dependency stages for parallel Strassen

Here M_i denotes the same product as P_i in Chapter 13. First prepare block sums/differences(a+d),(e+h),(c+d),(f−h),(g−e),(a+b),(c−a),(e+f),(b−d),(g+h) with cellwise parallel operations. Once inputs are ready, run seven products:

$$\begin{aligned} M_1 &= (a+d)(e+h), & M_2 &= (c+d)e, \\ M_3 &= a(f-h), & M_4 &= d(g-e), \\ M_5 &= (a+b)h, & M_6 &= (c-a)(e+f), \\ M_7 &= (b-d)(g+h). \end{aligned}$$

Sync after all seven, then combine in parallel: C11=M1+M4−M5+M7, C12=M3+M5, C21=M2+M4, C22=M1−M2+M3+M6. Products cannot consume unfinished sums, and combination cannot consume unfinished products.

Strassen's work, span, and parallel workspace

Seven half-size children and a constant number of matrix sums give

$$\begin{aligned} W_S(n) &= 7W_S(n/2) + \Theta(n^2) = \Theta(n^{\log_2 7}), \\ D_S(n) &= D_S(n/2) + \Theta(\log n) = \Theta(\log^2 n). \end{aligned}$$

Correctness follows from Chapter 13's expanded block identities and recursive induction, with the three additional parallel dependency stages. Parallelism Θ(n^(log₂7)/log²n) is smaller than for eight-way multiplication but work is lower. Maximizing a parallelism ratio alone is not the objective.

Sequential depth-first O(n²) temporaries do not transfer to simultaneous full child expansion. If each call keeps its own Θ(n²) workspace through join, eight-way space satisfies Q(n)=8Q(n/2)+Θ(n²)=Θ(n³); seven-way space is Θ(n^(log₂7)). Finite P and reuse can lower peaks, but require a consistent schedule/span analysis rather than an unimplemented space claim.

Executable specification of parallel Strassen

PADD(X, Y, s) allocates independent R and uses nested parallel loops to write R[i, j]=X[i, j]+s · Y[i, j], waiting before return. For s=±1, work is $\Theta(n^2)$, span $\Theta(\log n)$. Pad equal-order A, B to powers of two; the outer wrapper handles n=0.

```
P-STRASSEN(A, B):
    n=A.rows
    if n == 1: return [[A[0,0]*B[0,0]]]
    view A as [[a, b],[c, d]], B as [[e, f],[g, h]]
    S1=PADD(a, d,+1); S2=PADD(e, h,+1)
    S3=PADD(c, d,+1); S4=PADD(f, h,-1)
    S5=PADD(g, e,-1); S6=PADD(a, b,+1)
    S7=PADD(c, a,-1); S8=PADD(e, f,+1)
    S9=PADD(b, d,-1); S10=PADD(g, h,+1)
    M1=spawn P-STRASSEN(S1, S2)
    M2=spawn P-STRASSEN(S3, e)
    M3=spawn P-STRASSEN(a, S4)
    M4=spawn P-STRASSEN(d, S5)
```


Executable specification of parallel Strassen (continued)

```
    M5=spawn P-STRASSEN(S6, h)
    M6=spawn P-STRASSEN(S7, S8)
    M7=P-STRASSEN(S9, S10)
    sync
    allocate C[n, n], viewed as four blocks
    parallel for i=0 to n/2-1:
        parallel for j=0 to n/2-1:
            C11[i, j]=M1[i, j]+M4[i, j]-M5[i, j]+M7[i, j]
            C12[i, j]=M3[i, j]+M5[i, j]
            C21[i, j]=M2[i, j]+M4[i, j]
            C22[i, j]=M1[i, j]-M2[i, j]+M3[i, j]+M6[i, j]
    return C
```

Ten PADD calls are sequential here, but their fixed count retains $\Theta(\log n)$ preparation span. Finish all before spawning products, consistent with the three-stage contract. No arbitrarily long serial issuing chain is ignored.

Multithreaded merge sort

Parallel recursive sorts with a serial merge are insufficient:

```
PAR-SORT-SERIAL-MERGE(A, l, r):
    if r-l <= 1: return
    m=floor((l+r)/2)
    spawn PAR-SORT-SERIAL-MERGE(A, l, m)
    PAR-SORT-SERIAL-MERGE(A, m, r)
    sync
    serial MERGE the sorted halves using an auxiliary buffer
```

Work satisfies2W(n/2)+Θ(n)=Θ(n log n), but span D(n/2)+Θ(n)=Θ(n), giving only Θ(log n) parallelism. Sync before merge prevents reading unfinished runs. Reducing span further requires parallel merge itself.

Define half-open cuts and tie order first

Read-only T contains runs[l1, r1),[l2, r2). Output O[o, o+N), where N is total length, must not overwrite T. Compare records by(key, original position), a total order with unique original positions, so equal keys retain original order.

```
LOWER-BOUND(T, l, r, x):
    while l < r:
        m=floor((l+r)/2)
        if T[m] < x: l=m+1
        else: r=m
    return l
```

The original run's boundary between values<x and≥x remains in current[l, r]. Positions before current l are<x; positions at and beyond current r within the original run are≥x. Each comparison preserves a possible cut and strictly reduces interval length. An empty run returns l without reading nonexistent T[l]. Work and span are O(1+log(n2+1)), including constant empty handling.

Complete parallel merge, including empty runs

```
P-MERGE(T, l1, r1, l2, r2, O, o):
    n1=r1-l1; n2=r2-l2
    if n1 < n2: swap (l1, r1),(l2, r2); swap n1, n2
    if n1 == 0: return
    m=floor((l1+r1)/2)
    j=LOWER-BOUND(T, l2, r2, T[m])
    q=o+(m-l1)+(j-l2)
    O[q]=T[m]
    spawn P-MERGE(T, l1, m, l2, j, O, o)
    P-MERGE(T, m+1, r1, j, r2, O, q+1)
    sync
```

The longer run occupies the first descriptor. Swapping views neither moves nor relabels records. Empty first means both empty. Empty second still permits recursive median placement from the first without reading an empty interval. Each call removes one pivot, leaving total child size N−1, so recursion terminates.

General proof of position, correctness, and stability

There are m−l1 first-run and j−l2 second-run records smaller than the pivot, total q−o. Every remaining record is larger in the total order. The pivot therefore belongs at q. Child output ranges[o, q),[q+1, o+N) are disjoint from one another and from the pivot.

Induct on N. Empty merge is correct. Both children are smaller and correctly merge their designated records by induction. Concatenating left result, pivot, right result is sorted and contains every input record exactly once. T is read-only and output ranges have exclusive writers, so no determinacy race occurs. Sync makes the complete answer available.

Original positions order equal keys, proving stability. A key-only comparison with the same lower_bound rule after swapping longer-run roles can place an equal right-run record first. Correct key sorting alone does not prove stability.

Root cut and binary-search steps for eight records

Use T=[1a,4b,6c,9d | 2e,3f,6g,8h], where letters a..h identify original positions0..7. Compare keys first, then positions. Set l1=0, r1=4, l2=4, r2=8, o=0. The first length 4 run has m=2, pivot6c.

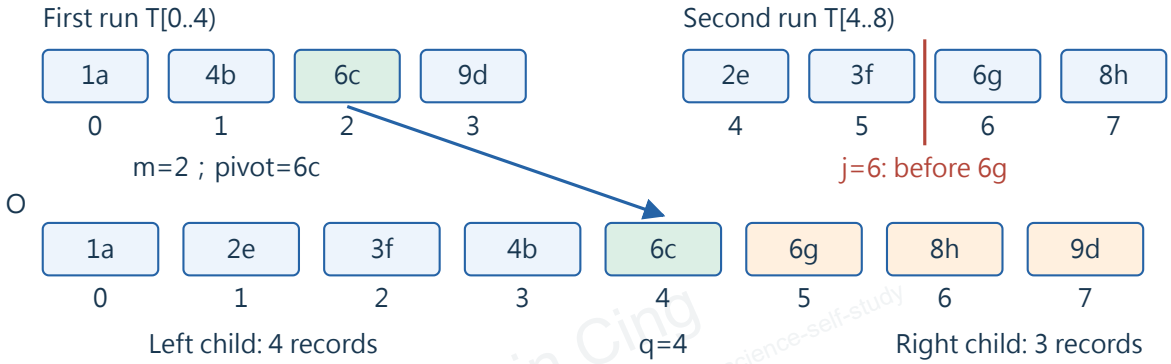
Search[l, r)	mid	Comparison with 6c	Update
[4,8)	6	6g>6c: equal key, later position	r=6
[4,6)	5	3f<6c	l=6
[6,6)	none	empty interval	return j=6

Position q=0+(2−0)+(6−4)=4. Left child contains 4 records, right child3.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Root cut and binary-search steps for eight records (continued)

Median pivot rank: $q=o+(m-l_1)+(j-l_2)=4$



Compare(key, original position): 6c<6g. Left, pivot, right outputs are disjoint.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A duplicate-key trace

Let `A=[1a,4b,4c,8d]`, `B=[2e,4f,5g,9h]`, with original positions b < c < f. Select `x=4c`. Two items precede it in A, and B's lower_bound position is 1, so `q=2+1=3`.

The left task merges `[1a,4b]` with `[2e]`; the right merges `[8d]` with `[4f,5g,9h]`. The combined output is `[1a,2e,4b,4c,4f,5g,8d,9h]`.

Median split and destination

A 1a 4b | 4c | 8d

B 2e | 4f 5g 9h

pivot 4c → output[3]

lower_bound in B = 1

left task: A[1a,4b] + B[2e]
→ [1a,2e,4b]

right task: A[8d] + B[4f,5g,9h]
→ [4f,5g,8d,9h]

left [0,3) pivot [3] right [4,8)

Every output write on the same example

One legal depth-first serialization writes pivots in the following order. Parallel interleavings may differ, but destinations stay fixed and each is written once.

Write	Pivot	q, zero-based	Meaning
1	6c	4	Root division
2	4b	3	Last record of root's left region
3	3f	2	Median after swapping longer run
4	2e	1	Small-run cut
5	1a	0	Other run empty
6	8h	6	Median after right-region swap
7	6g	5	Right region's left child
8	9d	7	Right region's right child

Final O=[1a,2e,3f,4b,6c,6g,8h,9d], with c still before g. Running the right child first gives the same answer because T is read-only and every destination unique. Sortedness does not rely on a global sequential write order.

Upper and lower bounds on worst-case merge span

For n1≥n2, either child has size at most⌊n1/2⌋+n2≤(N+n2)/2≤3N/4, even if the other cut lands at an endpoint. Binary search costs O(log(N+1)), so

$$D_M(N) \le D_M(3N/4) + O(\log(N+1)) = O(\log^2(N+1)).$$

Constant-factor shrinkage gives O(log N) levels with logarithmic sizes O(log N−cr); their arithmetic sum gives the square.

For a lower bound, merge interleaved runs[0,2,…,2m−2] and[1,3,…,2m−1], with power-of-two m. The left child retains the same equal-length interleaving structure, halving length each level. Its other-run median search takes Θ(log m) comparisons. Summing log m,(log m−1),… gives Ω($\log^2$m), hence worst-case Θ($\log^2$N). If the other run stays empty, search costs constant and span is only Θ(log N). Worst-case order is not exact cost for every input.

Writing each record once does not pay for searches

Every nonempty call removes a distinct pivot, so there are N nonempty calls. Each creates two children, for 2N+1 total calls including empty returns. The N output writes give Ω(N), but an O(N) upper bound must also pay each binary search.

A size-m node has nonrecursive cost at most c log(m+1). Divide it among its m input records, charging each c log(m+1)/m. A fixed record x is charged along one ancestor chain until chosen as pivot.

Count r levels upward from the last call containing x. The 3/4 child bound makes that ancestor's size at least(4/3)^r. Sum these charges below, including empty children, without taking log0 or falsely assuming both small children have size at least N/4.

Each record's total search charge is bounded

Function $g(u) = \log_2(u+1)/u$ decreases for u>0. The sign of its derivative numerator is u/(u+1)−ln(u+1)<0, using $\int_0^u (1+t)^{-1}dt > u/(1+u)$. Thus the charge r levels upward is at most

$$c\frac{\log_2((4/3)^r+1)}{(4/3)^r} \le c[1 + r\log_2(4/3)](3/4)^r.$$

Both Σq^r=1/(1−q) and Σr q^r=q/(1−q)² converge at q=3/4; the latter follows by differentiating finite sums and taking limits. Each record pays O(1), for O(N) total. The N+1 empty calls add constant work each. With the output lower bound, W_M(N)=Θ(N).

This proof covers arbitrary legal cuts, empty runs, and unbalanced small children. A finite example is not the upper-bound proof.

Complete two-buffer parallel merge sort

Attach original positions to records and initialize separate arrays X, Y of length n to the same input, using one parallel copy. Call SORT(X, Y,0, n), then read Y.

```
SORT(src, dst, l, r):
    if l == r: return
    if r-l == 1: dst[l]=src[l]; return
    m=floor((l+r)/2)
    spawn SORT(dst, src, l, m)
    SORT(dst, src, m, r)
    sync
    P-MERGE(src, l, m, m, r, dst, l)
```

Children swap buffer roles, leaving their sorted runs in the parent's src. Sync before merging into dst. P-MERGE input and output must be different physical buffers. Two global buffers avoid allocating a new length-n temporary per call and avoid overwriting unread input.

Allocation and practical cutoffs

The bound assumes constant-time reservation of uninitialized buffers, with actual reads and writes charged separately. If allocation serially initializes n entries, that initialization alone can add Θ(n) span, invalidating the bound above.

The simple recursive version may hold Θ(n) buffer locations at each of O(log n) levels, for O(n log n) extra space. Two preallocated n-element buffers alternating roles between levels can reduce extra space to O(n), after separately proving that concurrent accesses do not conflict.

For small subproblems, a serial merge/sort cutoff reduces scheduling overhead. It changes constants and span and must be analyzed for the chosen cutoff; theoretical bounds are not benchmark results.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Buffer-swap correctness and race checks

At each SORT entry, both buffers' current intervals still contain that interval's original input. The root copy establishes this. Before launching children, the parent has not altered its interval, and each child touches only its disjoint half, so the entry property holds recursively.

A singleton copies to dst. By induction, children leave two sorted unchanged-multiset runs in parent src. After sync, P-MERGE stably merges them to dst. The parent need not promise src equals dst on exit; the equality condition is needed only on entry.

Even with role swaps, children write disjoint index ranges of the same two arrays. Parent merging waits for both. Data workspace is O(n), peak task descriptors O(n), and each call chain O(log n). Initial copying takes Θ(n) work,Θ(log n) span, without changing sorting orders.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A tight sort bound needs one compatible worst-case input

Every sort level has Θ(n) merge work and there are Θ(log n) levels, so W_S(n)=Θ(n log n). Span obeys

$$D_S(n) \le D_S(\lceil n/2 \rceil) + O(\log^2 n) = O(\log^3 n).$$

For the lower bound, the same input must force expensive merges at all relevant levels. For n=2^L, place keys in L-bit-reversed index order; for n=8, use[0,4,2,6,1,5,3,7]. Root halves sort into interleaved even and odd runs. Each child's relative ranks have the same construction. Every level therefore has an Ω(log²(size)) merge, and parent merging starts only after both children finish.

Along a child chain,Σ_(r=1..L)r²=Θ(L³), giving worst-case Θ(log³n) for powers of two. Average parallelism is Θ(n/log²n). Merge's squared-log span is not the sort's span.

Compatible expensive levels for nonpowers of two

For any n, recursively assign relative ranks. If n is even, put even ranks in the first half and odd in the second. If odd, put odd ranks in the shorter first half and even in the longer second. Recursively arrange each half's ordered key set the same way. Every sorted pair of runs interleaves with length difference at most1. For powers of two, this is bit reversal.

At each such merge, the longer-run median has m same-run records on its left, and m or m+1 other-run records. The left child remains interleaved with length difference at most1 and total size between N/2−1 and N/2+1. For N≥8, both runs have Θ(N) size. The selected path has Θ(log N) levels and logarithmic search cost at each shrinking scale, giving Θ(log²N) span.

The same recursive rank arrangement gives this property at every sort node. Follow the larger child, whose size roughly halves. Its completion precedes parent merge, so critical paths concatenate and give Ω(log³n). This supplies the tight lower bound for arbitrary n, without choosing incompatible worst inputs separately at different levels.

Compare work and worst span in one model

Algorithm	Work	Worst span	Average parallelism
Recursive P-FIB	$\Theta(\varphi^n)$	$\Theta(n)$	$\Theta(\varphi^n/n)$
Row-parallel matrix-vector	$\Theta(n^2)$	$\Theta(n)$	$\Theta(n)$
Reduction matrix-vector	$\Theta(n^2)$	$\Theta(\log n)$	$\Theta(n^2/\log n)$
Two outer parallel loops for matrix product	$\Theta(n^3)$	$\Theta(n)$	$\Theta(n^2)$
Eight-way recursive matrix product	$\Theta(n^3)$	$\Theta(\log^2 n)$	$\Theta(n^3/\log^2 n)$
Parallel Strassen	$\Theta(n^{\log_2 7})$	$\Theta(\log^2 n)$	$\Theta(n^{\log_2 7}/\log^2 n)$
DOT-reduction matrix product	$\Theta(n^3)$	$\Theta(\log n)$	$\Theta(n^3/\log n)$
Parallel merge	$\Theta(n)$	$\Theta(\log^2 n)$	$\Theta(n/\log^2 n)$
Parallel merge sort	$\Theta(n \log n)$	$\Theta(\log^3 n)$	$\Theta(n/\log^2 n)$

Asymptotic entries assume sufficiently large n; empty and singleton cases use explicit bases. Work efficiency, small span, and low space are distinct objectives. For fixed P, compare W/P, D, and memory together rather than substituting one parallelism ratio for the full analysis.

References and attribution

Cormen, Leiserson, Rivest, Stein, Introduction to Algorithms, third edition, Chapter 27. MIT 6.046J Fall 2004, Lecture 3, matrix multiplication and Strassen. MIT 6.046J Fall 2005, A Minicourse on Dynamic Multithreaded Algorithms, Handout 29.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study